AIM- To implement inter-task communication using a FreeRTOS queue where one task generates or acquires sensor data and another task receives the data.

APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, Ultrasonic Sensor, STM32 CUBE IDE, STM32 CubeMX.

THEORY

The implementation of inter-task communication in a real-time operating system (RTOS) like FreeRTOS is a cornerstone of modular embedded design. While semaphores and task notifications are excellent for synchronization (signaling *that* an event happened), they are insufficient for transferring actual data. To solve the challenge of passing complex information—such as sensor readings, strings, or structured data—between independent execution threads, FreeRTOS provides the **Queue** mechanism.

The Anatomy of a Queue

A FreeRTOS queue is a thread-safe, kernel-managed buffer that operates on a **First-In, First-Out (FIFO)** principle. Unlike a standard C array or a global variable, a queue is designed for concurrency. It consists of a fixed number of "slots," each of a fixed size. For instance, if you are acquiring data from an RFID reader, you might define a queue with 10 slots, each large enough to hold a 32-bit integer or a custom `struct`.

One of the most critical features of the FreeRTOS queue is that it uses **Copy by Value**. When a task "Sends" data to a queue, the data is physically copied into the queue's memory. This means the original variable can be modified or even go out of scope without corrupting the data waiting in the queue. While this consumes a small amount of CPU time for the copy operation, it significantly simplifies memory management and prevents "race conditions" where two tasks try to access the same memory address simultaneously.

Thread Safety and Blocking Logic

In a multi-tasking environment, a queue must handle situations where a producer is faster than a consumer (Queue Full) or a consumer is faster than a producer (Queue Empty). FreeRTOS handles this through **Blocking**.

- **The Consumer (Receiver):** When a task attempts to read from an empty queue, it can enter the **Blocked** state. It remains in this state—consuming zero CPU cycles—until another task or an Interrupt Service Routine (ISR) places data into the queue. The moment data arrives, the scheduler moves the receiver to the **Ready** state.
- **The Producer (Sender):** Conversely, if a queue is full, the sending task can block until space becomes available. This creates a natural "back-pressure" system that synchronizes the speed of data generation with the speed of data processing.

The Producer-Consumer Model

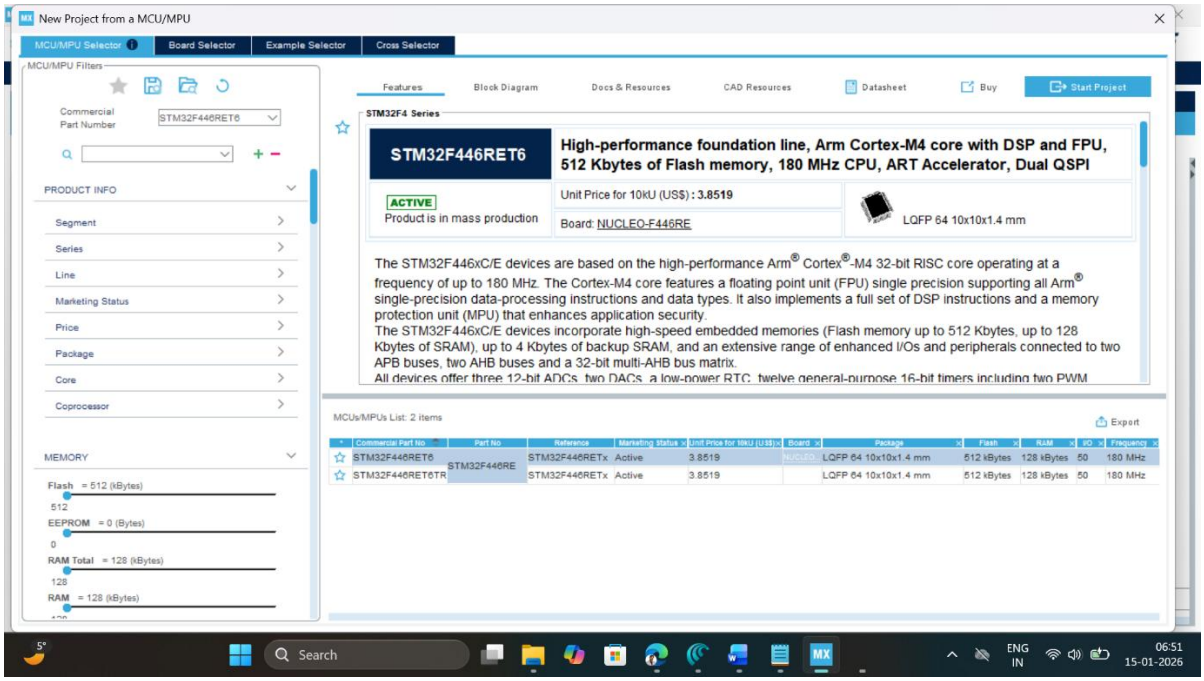
This experiment follows the **Producer-Consumer** architectural pattern.

1. **The Producer (Sensor Task):** This task interfaces with hardware (e.g., an Ultrasonic sensor, ADC, I2C sensor or RFID module). Once it acquires a valid reading, it calls `osMessageQueuePut`.
2. **The Consumer (Display/Processing Task):** This task sits in a loop calling `osMessageQueueGet`. It remains dormant until a new sensor value is available, at which point it wakes up to process the data (e.g., displaying it on an LCD or making a logic decision).

The producer task in this experiment involves reading the distance with a period of 1000 ms.

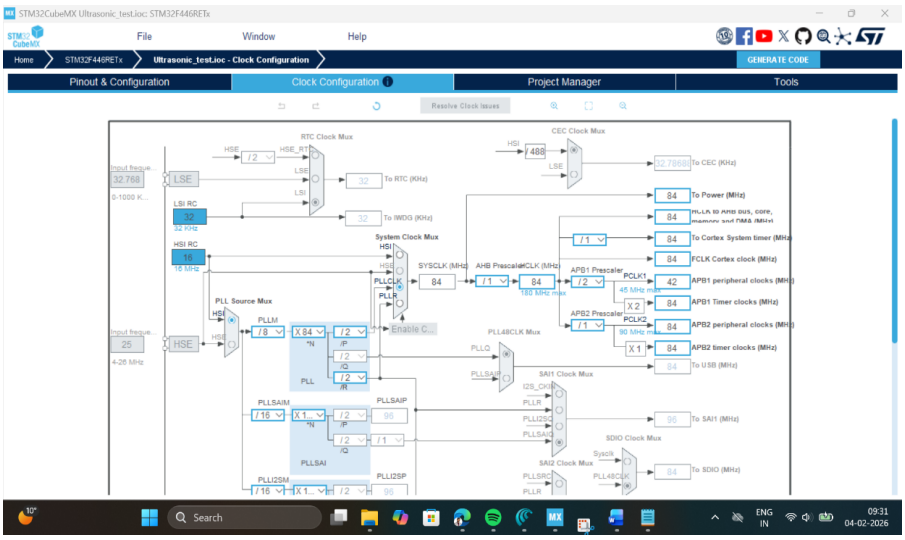
PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.

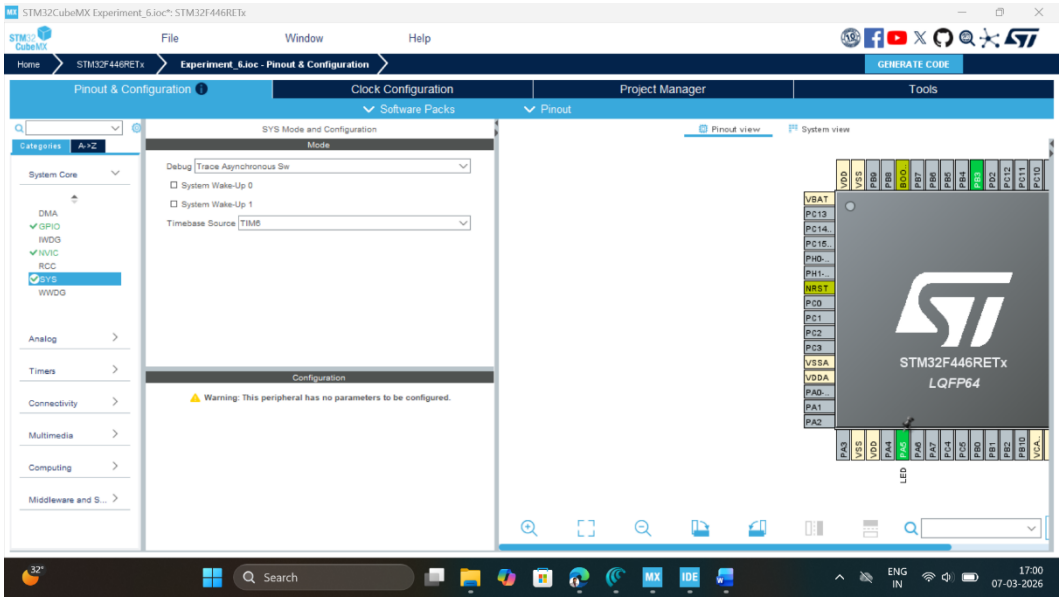


2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.

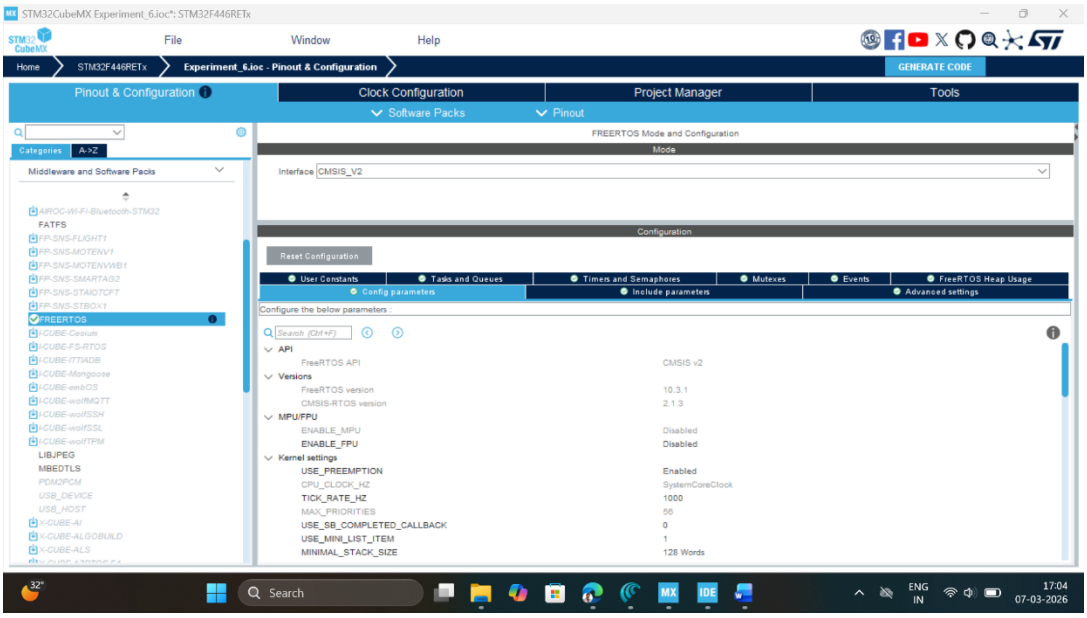
3. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



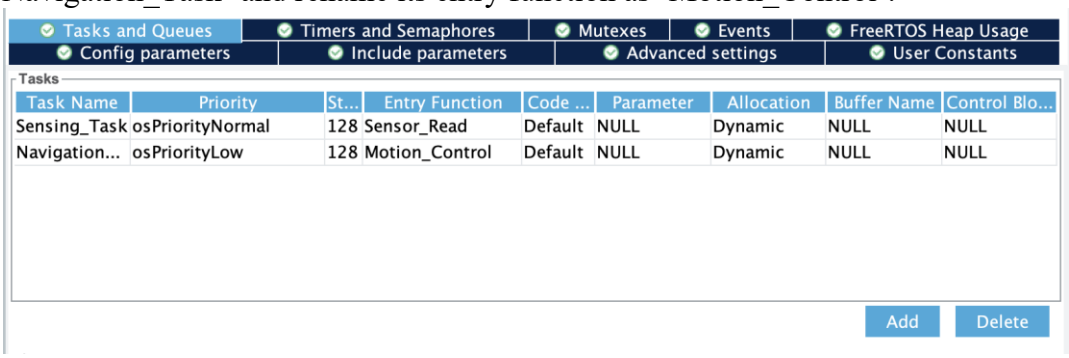
4. **SYS Mode:** Click on SYS mode and select debug as *Trace Asynchronous Sw*, and Timebase source to **TIM6.**



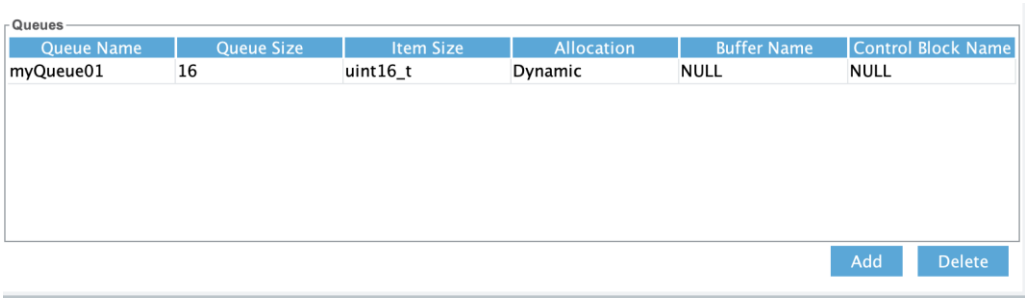
5. **FreeRTOS Configuration:** Click on Middleware and Software Pack, then select FreeRTOS software pack, which is already available with STMCube Mx. Select interface as **CMSIS_V2.**



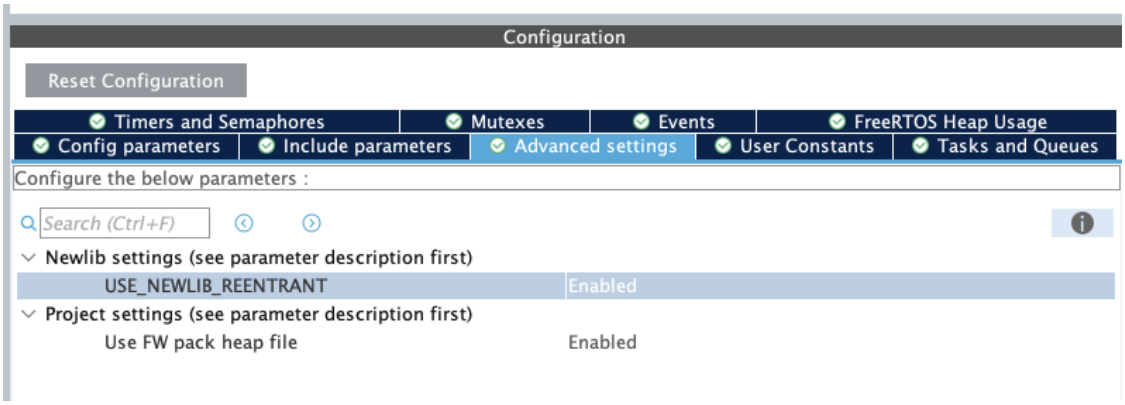
6. Click on Task & Queue Tab of configuration settings and rename the default task as ‘Sensing_Task’. Rename the entry function as ‘Sensor_Read’. Add another task named ‘Navigation_Task’ and rename its entry function as ‘Motion_Control’.



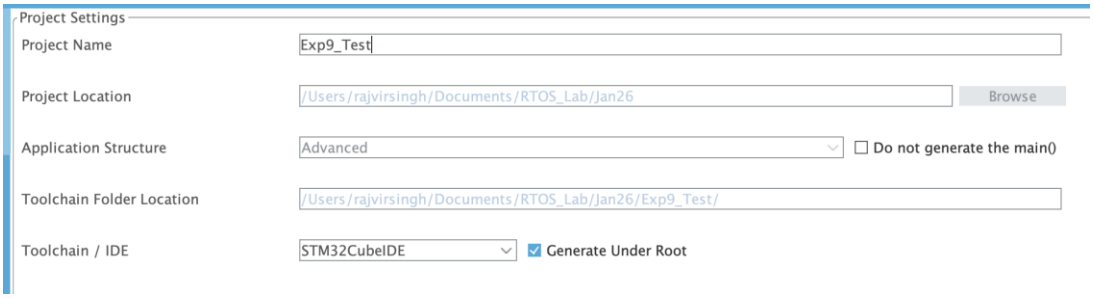
7. Now ‘Add’ a Queue to the application by clicking the Add button as shown below:



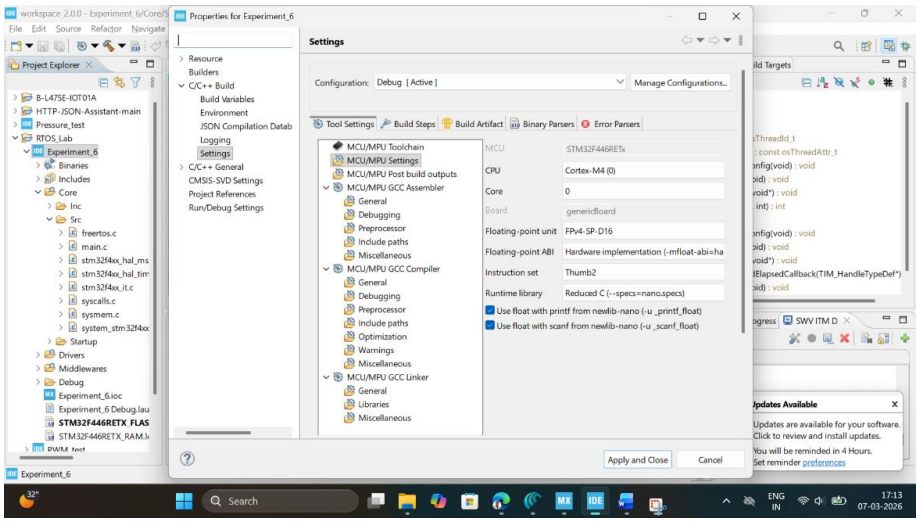
8. Select Advanced Settings tab, and enable the USE_NEWLIB_REENTRANT setting



9. Write Project name and select IDE.

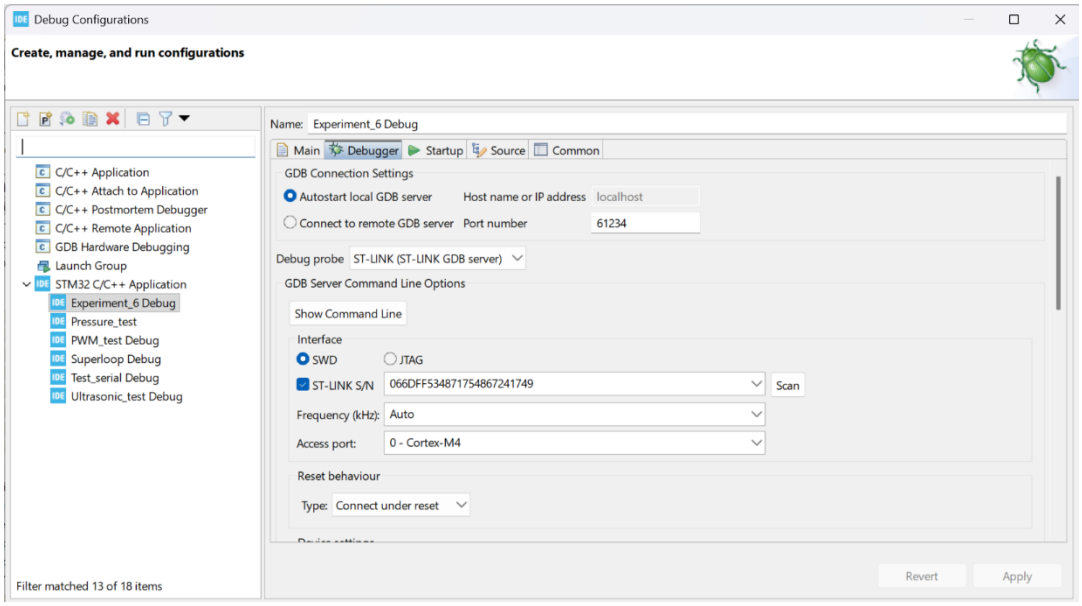


10. Click on generate code.
11. Click on the open project.
12. Click ‘OK’ on the generated popup showing “successfully imported the project on the workspace” and the project will be successfully added to the cube IDE.
13. Open the main source code on Cube IDE by clicking on **main.c**
14. Click on project properties, go to C/C++ Build settings check box for *printf* and *scanf* settings.



15. Now click on Debug icon and configure debugger for SWV ITM Trace

16. Click on Debugger configuration, select the debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up ST-LINK id.



17. Next, select the Serial Wire Viewer (SWV) and check the enable button and make sure to pass on the right core clock frequency which is set-up at point 4 here i.e. 84MHz. Then click on the Apply button and close this window.
18. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling./* *Private includes* ----------*/

```
/* USER CODE BEGIN Includes */
#include<stdio.h>
/* USER CODE END Includes */
```

Write the function to enable SWV ITM Console to trace the output of tasks

```
/* USER CODE BEGIN 0 */
int _write(int file, char *ptr, int len) {
for (int i = 0; i < len; i++) {
ITM_SendChar(*ptr++);
}
return len;
}
/* USER CODE END 0 */
```

Edit the following line of code to create a queue to store unsigned integers

```
/* Create the queue(s) */
 /* creation of myQueue01 */
 myQueue01Handle = osMessageQueueNew (16, sizeof(unsigned int),
&myQueue01_attributes);
 /* USER CODE BEGIN RTOS_QUEUES */
 /* add queues, ... */
 /* USER CODE END RTOS_QUEUES */
```

Edit the Sensor_Read task function as given below

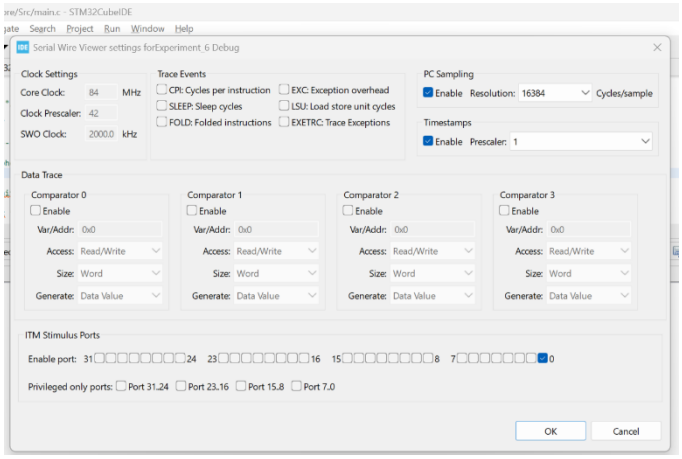
```
void Sensor_Read(void *argument)
{
 /* USER CODE BEGIN 5 */
        unsigned int dist = 0;
 /* Infinite loop */
 for(;;)
 {
          printf("Inside Data Producer Task\n");
          dist = dist + 1;
          osMessageQueuePut(myQueue01Handle, &dist, 0, osWaitForever);
          osDelay(1000);
 }
 /* USER CODE END 5 */
}
```

Edit the Motion_Control task function as given below

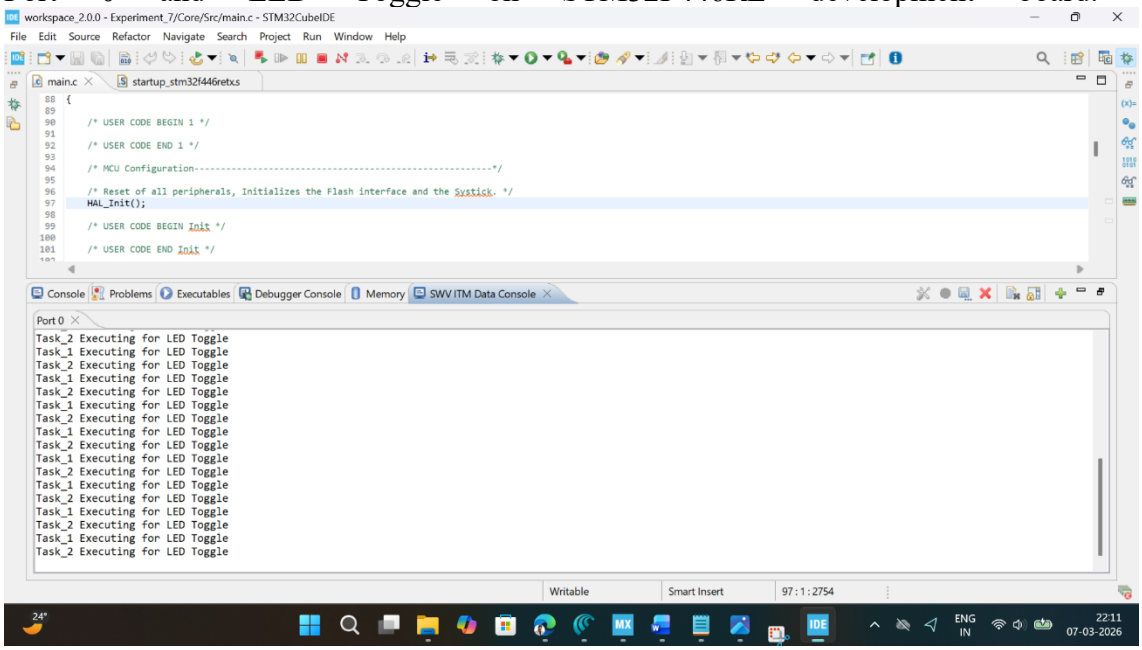
```
void Motion_Control(void *argument)
{
 /* USER CODE BEGIN Motion_Control */
        unsigned int distance;
 /* Infinite loop */
 for(;;)
 {
          printf("Inside Data Consumer Task\n");
```

```
        osMessageQueueGet(myQueue01Handle, &distance, NULL, osWaitForever);
        printf("Distance is %u\n",distance);
    }
/* USER CODE END Motion_Control */
}
```

19. Now, click on Window tab, select show view other select SWV ITM Data Console.
20. Now click on the Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.
21. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



22. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program. Now you can see the message on Port 0 and LED Toggle on STM32F446RE development board.



23. Start the debug session and use the SWV ITM Data Console to see the trace output from the tasks.

.

OBSERVATION:

Observation Number	Query	Response	
1	Specify the priority level of the following tasks by looking up from the Cube MX:	Sensor_Read	Normal Priority
		Motion_Control	Normal Priority
2	Select the correct option based on the Cube MX priority setting for both tasks in the application.	Both tasks have same priority Sensor_Task has higher priority Motion_Task has higher priority	
3	From the trace output printed in 'SWV ITM Data Console', identify which task is running more frequently?	Sensor task Motion Control task Both are running alternately	
4	Is the consumer task able to receive every value produced by the producer task?	Yes No	

RESULT

The experiment successfully demonstrated synchronized inter-task communication using a **FreeRTOS Message Queue** within a producer-consumer architecture. The **Sensor_Read** task functioned as the producer, incrementing a distance variable and utilizing `osMessageQueuePut` to copy the data into the kernel-managed buffer every 1 second. Simultaneously, the **Motion_Control** task acted as the consumer, remaining in a power-efficient **Blocked** state via `osMessageQueueGet` until data became available.

Reflection Summary

1. Elaborate the meaning of the term 'thread-safe' in context of the following statement- 'A FreeRTOS implementation of Queue is thread-safe'.

   A FreeRTOS Queue is called **thread-safe** because multiple tasks (threads) can access the same queue safely without causing data corruption or unpredictable behavior.
   In FreeRTOS, synchronization mechanisms are internally used so that when one task is writing data into the queue and another task is reading data from the queue at the same time, the queue still works correctly.
   This means:
   - Two tasks cannot modify queue data simultaneously in a harmful way.
   - Race conditions are avoided.
   - Data integrity is maintained.
   - Tasks automatically wait (block) when the queue is full or empty.

   For example:
   - If a producer task tries to send data into a full queue, it is blocked until space becomes available.
   - If a consumer task tries to read from an empty queue, it waits until new data arrives.

   Thus, FreeRTOS Queues provide safe communication between multiple concurrent tasks an RTOS environment.

2. Develop and test an implementation that uses a Queue to synchronize a fast data producing task with a slow data consuming task? The fast data producing task will quickly fill the queue to its full capacity after which the data production should stop till the queue can accommodate data.

Aim

To synchronize a fast data-producing task with a slow data-consuming task using a FreeRTOS Queue.

Theory

In this implementation:

- The producer task generates data very quickly.
- The consumer task processes data slowly.
- Since the producer is faster, the queue becomes full rapidly.
- Once the queue reaches maximum capacity, the producer task automatically blocks until the consumer removes some data.

This demonstrates flow control and synchronization using FreeRTOS Queues.

Program Logic

Producer Task

- Generates integer data continuously.
- Sends data to queue using xQueueSend().
- Gets blocked automatically when queue becomes full.

Consumer Task

- Reads data slowly using xQueueReceive().
- Introduces delay to simulate slow processing.

Expected Observation

- Queue fills quickly.
- Producer stops automatically when queue is full.
- Consumer slowly removes data.
- Producer resumes again after free space becomes available.

Result

The experiment successfully demonstrated synchronization between a fast producer task and a slow consumer task using a FreeRTOS Queue. The queue prevented overflow by blocking the producer task whenever the queue became full.

3. Develop and test an implementation that uses a Queue to synchronize a slow data producing task with a fast data consuming task? The fast data consuming task will quickly empty the queue after which the data consumption should stop till the data becomes available in the queue.

Aim

To synchronize a slow data-producing task with a fast data-consuming task using a FreeRTOS Queue.

Theory

In this implementation:

- The producer generates data slowly.
- The consumer consumes data very quickly.
- The queue becomes empty rapidly.

- Once the queue becomes empty, the consumer task automatically blocks until new data becomes available.

This demonstrates synchronization and proper task coordination using FreeRTOS Queues.

Program Logic

Producer Task

- Produces data slowly using delay.
- Sends data into queue periodically.

Consumer Task

- Continuously checks queue for data.
- Reads data immediately.
- Gets blocked when queue becomes empty.

Expected Observation

- Consumer empties queue quickly.
- Queue frequently becomes empty.
- Consumer waits for new data.
- Producer periodically wakes consumer by inserting data.

Result

The experiment successfully demonstrated synchronization between a slow producer task and a fast consumer task using a FreeRTOS Queue. The queue prevented underflow conditions by blocking the consumer task whenever the queue became empty.